Before / After: AGENTS.md on Issue #5

HW3 Context Engineering · TakeoutPick (P2) · Ruoyu Wang · Oct 30, 2026

1. Result in one table

The same AI agent implemented the same Sprint 1 issue three times from a clean start. Only the rules file changed. Each result was scored by the same script (34 objective checks).

	A — no rules file	B — AGENTS.md v1.0	C — AGENTS.md v1.1
Total score (34 checks)	**28 / 34 (82%)**	**32 / 34 (94%)**	**33 / 34 (97%)**
PRD acceptance criteria	7 / 7	6 / 7 (API skipped)	7 / 7
Architecture & naming	9 / 12	12 / 12	12 / 12
Test quality	3 / 6	5 / 6	5 / 6 (1 justified)
Accessibility checks	2 / 2	2 / 2	2 / 2
Process (branch, commits, TDD)	3 / 3	3 / 3	3 / 3
Project-rule violations	6	0	0
New test cases	34 (6 with mocked DB)	32	42 (7 real-DB integration)
Badge text contrast (WCAG ≥ 4.5)	3.6 – 4.4 : 1 ✗	3.3 – 4.4 : 1 ✗	5.4 – 5.6 : 1 ✓
lint / typecheck / unit tests	pass	pass	pass

Bottom line: the v1.0 rules file removed every project-rule violation (6 → 0) but made the agent drop one acceptance criterion. The changes in v1.1 fixed that, and the final run met all 5 criteria with no violations.

2. Methodology

Item	How it was controlled
Feature	Issue #5 — “Show deal status and price after deal with a minimum-order hint” (US3, Sprint 1, 5 acceptance criteria).
Repository	Three byte-identical copies of the P2 repo (PRD, UI spec, schema, money/http helpers, configs). Run A: `AGENTS.md` deleted. Run B: `AGENTS.md` v1.0. Run C: `AGENTS.md` v1.1 plus the two repo fixes v1.1 depends on (darker color tokens, test cleanup).
Agent	Same model, same harness, a brand-new conversation per run (no memory of earlier runs or of this report).
Prompt	Identical one-paragraph request in all runs. B and C add exactly one sentence: “this IDE auto-loads AGENTS.md … read it first and follow it” — which is what an IDE does with a rules file.
Environment	Same for all: dependencies installed, no database, no network or GitHub access.
Scoring	`experiment/measure.sh`: runs lint, typecheck, unit tests and coverage, then 34 PASS/FAIL checks taken from the PRD’s acceptance criteria and the rules. Full output in `experiment/*-score.txt`; code in `experiment/A…`, `B…`, `C…` (each is a git repo: `git log`, `git diff HEAD~2`).

Fairness note: run A was not starved of context. The PRD, UI spec and PR template were in the repo, and A found and read them. So this test measures what the **rules file itself** adds on top of good documentation. That is why A still scores 82%.

3. Specific differences, with code

3.1 Naming conventions and architecture

Without rules the agent used the common React habit of PascalCase file names. It also put database access straight into the route handler. With rules it followed the project layout: kebab-case files and route → service → domain.

	A — no rules	B / C — with rules
Component files	`DealBadge.tsx`, `PriceWithDeal.tsx`, `MinOrderHint.tsx`	`deal-badge.tsx`, `price-with-deal.tsx`, `min-order-hint.tsx`
Test files	one catch-all `deals.test.tsx` for 3 components	one test file beside each source file
Prop types	`export interface …Props`	`type …Props = { … }`
API route	route imports db and builds the Prisma query itself	C: `route.ts` (8 lines) → `deal-service.ts` → domain; zod schema in `lib/validation/deal.ts`

A — src/app/api/v1/deals/route.ts (route talks to the database directly)

```
import { db } from "@/lib/server/db";
export async function GET(request: Request): Promise<Response> {
  ...
  const now = new Date();
  const deals = await db.deal.findMany({ where: { hidden: false, ... }, ... });
```

C — src/app/api/v1/deals/route.ts (thin handler; only the service touches db)

```
const query = parseOrThrow(listDealsQuerySchema, params);
// Read the clock once per request and pass it down (AGENTS.md §1.3).
const deals = await listDeals({ ...query, now: new Date() });
return ok(deals);
```

3.2 Hidden clock in “pure” domain code

The PRD calls deal status time-dependent, and the rules say domain functions get `now` as a parameter. Without rules the agent added a default `now = new Date()`. That hides the clock. It also let one test and the components fall back to the real time, which breaks under Next 16 Cache Components.

A — src/lib/domain/deals.ts

```
export function dealStatus(expiresAt: Date | string, now: Date = new Date()): DealStatus {
// ...and in deals.test.ts the real clock leaks in:
expect(dealStatus(new Date(Date.now() + 100 * HOUR))).toBe("ACTIVE");
```

B — src/lib/domain/deals.ts

```
export const EXPIRING_SOON_HOURS = 72;
/** Expired when now ≥ expiresAt; expiring soon when 72 h or less are left (the boundary itself
counts). */
export function getDealStatus(expiresAt: Date, now: Date): DealStatus {
```

3.3 Understanding the design / mockup intent

All three runs read `docs/ui-spec.md`. They used the `DealBadge`, `PriceWithDeal` and `MinOrderHint` component names, the exact hint text "Add $X.XX more to use this deal" and the `deal-*` color tokens. They all paired color with a word and an icon. Differences:

- **Data flow:** A's `DealBadge` calculates its own status from `expiresAt` (and the clock). B and C's badge gets `status` as a prop, as the rules ask ("components never read the clock").
- **Contrast:** A dimmed the Expired pill with `opacity-70`. B used grey text on a light grey background. Measured, both are 3.3–3.6 : 1, below WCAG AA. The cause was the v1.0 tokens, which failed AA themselves (white on `deal-active` = 4.38 : 1). The rule said "≥ 4.5 : 1" but neither agent could check that. v1.1 darkens the tokens and lists the approved pairs. C's badge measures 5.4–5.6 : 1.
- **Credit to A:** it added screen-reader text ("Was … now …") around the struck-through price, which B did not. This is now a candidate rule for Sprint 2.
- **Icons:** the rules prefer `lucide-react`, but it was not installed. A and B used text glyphs (✓ ⏳ ✕). C followed the new v1.1 fallback: an inline SVG with `aria-hidden` and a `TODO(#5)`.

3.4 Quality of the tests

Check	A	B	C
Boundary cases (exactly 72 h, at expiry, subtotal = minimum, amount-off > subtotal, 0 % / 100 %)	yes	yes	yes
Fixed dates only (no real clock)	no — `Date.now()` in domain test	yes	unit: yes · route int-test: relative to now (documented)
Database	Prisma mocked with `vi.mock`, plus fake timers	no API tests	7 tests against real Postgres
Test names follow the AC	partly	yes	yes
Coverage of src/lib/domain (lines / branches)	100 % / 94 %	100 % / 89 %	100 % / 92 %

A's route test passes, but because it mocks the database it would also pass with a wrong Prisma `where` clause. C's integration tests really run the query (they need Docker; see the caveats).

3.5 Where rules v1.0 made things worse: scope

Run B skipped AC5 (`GET /api/v1/deals?status=active`). Its own words: "It needs a deal service, a route handler and integration tests against real Postgres, and no database is running here. It's also outside the feature as you described it." Two v1.0 rules caused this: "Do not add features … the issue does not ask for", and "integration tests use a real database". Together, and without access to the GitHub issue, they made the agent treat the short chat request as the whole scope. Run A had no such rules and simply built the route.

3.6 Workflow

All three runs created `feature/5-deal-status`, used `test(#5):` / `feat(#5):` commits, and committed failing tests first. A did too, because the PR and issue templates in `.github/` mention the same process. B

and C reported each AC as checked or unchecked and listed the open Definition-of-Done items (screenshots, PR, moving the board card). A did not.

4. What changed in the rules (v1.0 → v1.1 → v1.2)

Finding in the test	Rule added or changed
B dropped AC5 (no DB, short prompt)	§0: scope = every AC of the issue (PRD §5 if GitHub is unreachable); never drop one silently; missing infrastructure → still write the service and integration test, report "written, not run".
A: `now = new Date()` defaults	§1.3: now is a required parameter; read the clock once per request; components never read it.
A: PascalCase files, mixed interface/type	§1.4: kebab-case also for component files; `type` for props; service naming.
A: mocked db, fake timers; both: missing cleanup	§1.6: one test file per source file; no db mocks or fake timers; `vitest.setup.ts` now runs `cleanup()`.
Glyph icons, direct db in route	§4: preferred libraries are pre-approved, SVG fallback if offline; every route goes through a service (example given for #5).
Both badges below 4.5 : 1	§6.2 + `globals.css`: tokens darkened, list of verified color pairs, no `opacity` on text.
C had to pick between two rules	v1.2 §1.6: route-handler integration tests may seed data relative to the real time, 1 h from any boundary.

Verification: run C used the v1.1 file with the same prompt. It met all 5 AC, kept 0 rule violations, used the service layer, and added 7 real-database integration tests. Score: 33 / 34. Its one failed check is the conflict that v1.2 resolves.

5. Caveats

- One run per condition. AI output varies between runs, so treat the numbers as indicative, not statistically proven.
- I wrote the checks from the PRD and the rules. A could not score on rules it never saw. That is the point of the test, but it favours B and C on the convention checks. The acceptance-criteria and test-quality checks are rule-independent.
- C's 7 integration tests are written and fail only because no `DATABASE_URL` is set. They have not yet run against Postgres (`docker compose up -d db && npx prisma migrate dev && npm run test:int`).